

AI AGENTS / AI ENGINEERING / SOFTWARE TESTING

Shipping code without human verification

Discover how automated verification allows engineering teams to safely ship AI-generated code without human review bottlenecks.

Jul 29th, 2026 9:00am by [Jade Rubick](#) and [Ankit Jain](#)



Aakash Dhage for Unsplash+

[Aviator](#) sponsored this post.

Agents are writing code faster than humans can review it. The answer is not “review faster”; that would be like building a faster horse instead of a car.

Every engineering leader will have to answer one question soon: how much code can you ship to production without a human looking at it? Teams that keep verifying every line will be buried and **become the bottleneck**.

Maybe you start with one percent. What would it take to get some portion of your commits to go out to production without any human verification? It’s analogous to CD, where the thinking was: What is the set of practices we need to safely ship something into production as soon as it’s approved?

that the change behaves as intended and that the software quality meets the organization's standards.

For example, a new API should have automated testing to make sure it is correct. It should also have automated verification to ensure that the API is structured according to your practices, scales well, and uses your standard set of error codes, and so on.



Aviator is a developer productivity platform used by modern engineering teams to ship AI-generated code at scale. Shared context, faster reviews, deterministic verification, and merge-to-deploy automation built for the volume AI creates.

[Learn More](#) →

THE LATEST FROM AVIATOR

[Move code review before the code](#)

3 August 2026

[AI Code Verification vs AI Code Review: What's the Difference](#)

23 July 2026

[How to kill the code review](#)

22 July 2026

HEAR MORE FROM OUR SPONSOR

svo1975pvn1982@gmail.com

Submit

These are related but separate quality gates. Traditionally, they have been joined into the practice of code review. Review is where knowledge moves through a team. The author has to explain what they did and why, and the reviewer learns how the team is changing.

the afternoon, and it scales only with human time. Catching bugs this way was always inefficient, and it never offered strong guarantees.

TRENDING STORIES

1. [Shipping code without human verification](#)
2. ["Time to clean up human slop": Why AI now reviews code better than your teammate.](#)
3. [The 800 mistakes that could reshape Meta's AI coding strategy](#)
4. [Why a 'Boring' Database Is Your Secret AI Superpower](#)
5. [How to kill the code review](#)

We're moving toward "human on the loop"

The basic loop for agentic tools is:

- **Provide context**
- Have it do some work
- Evaluate that it's successful
- Improve the loop if you're not satisfied with the result.

Humans are moving from doing the work themselves to overseeing agents doing the work to being a **human ON the loop**. In the loop means personally reviewing and approving each change. On the loop means maintaining the system that does the reviewing and stepping in when the system itself needs work.

"Instead of each of us contributing individually, we're maintaining the system of delivery. We're maintaining a factory, rather than acting as artisans."

example,

-
- A security engineer might oversee a set of security tooling that validates the security of generated code and looks for the release of PII data and secrets.

This pattern is often referred to as moving to a factory model of software. Instead of each of us contributing individually, we're maintaining the *system* of delivery. We're maintaining a factory, rather than acting as artisans.

Agents need feedback fast

What does this look like for testing and for verification?

One approach could be the role of an automated verification engineer, a cross between a developer experience engineer and quality assurance. If done right, automated verification could become the highest-leverage work an engineering team can invest in.

Agents rarely land the right result on the first pass; therefore, agents need stronger guardrails than humans ever did. It's much more likely they bash their heads against a set of constraints and iterate against feedback until they come up with something that works well. The speed and reliability of that feedback govern how good the output gets. A fast, trustworthy signal on what a change just broke is worth more to an agent than almost anything else you can hand it.

"A fast, trustworthy signal on what a change just broke is worth more to an agent than almost anything else you can hand it."

Improving a coding agent's ability to detect test and verification failures automatically by 50% may speed up the engineering organization by 200% because a much larger percentage of its changes will be able to be accepted. Automated verification speeds up the company and improves quality.

YOUTUBE.COM/THENEWSTACK

Tech moves fast, don't miss an episode. Subscribe to our YouTube channel to stream all our podcasts, interviews, demos, and more.

SUBSCRIBE

What is an automated verification engineer?

QA, in its traditional form, was a downstream process. Engineering handed work off, QA found problems, and work bounced back. When Jade surveyed a group of engineering leaders about the role of QA in startups, the response was near-uniform: the handoff was ineffective, and most would not run it that way again. The failure was the quality gate sitting after engineering rather than inside it, which let engineers treat quality as someone else's responsibility.

Before AI, in a lot of startup circles, the solution was either 1) to shrink or eliminate QA, or 2) to move them more in line with the engineering team.

The automated verification engineer (AVE) is really an experiment. The idea is to create a role that is completely focused on automating both testing and verification. So let's spell out what this experiment might look like:

Testing responsibilities

- Focus one layer above the automated tests—they oversee the “test harness.” This is a set of rules that enforces broad test coverage of all work. The test harness encourages deep, adversarial quality checks. It requires extensive unit and UI testing for a PR to pass to production.

and are responsible for keeping the tests running fast.

Verification responsibilities

- Responsible for a verification harness. This is a set of context, LLM tooling, and deterministic tooling that verifies that every PR delivers the intended outcome and meets your company's invariants for what a PR should deliver. It should follow all your practices, have observability baked in, and meet all your standards.
- Perhaps the verification harness could include test plans as well. They won't be perfect at first, but flaws in the plan are seen as things to improve, rather than editing a template test plan every time.
- Verification failures are seen as a chance to improve that verification harness.

The shape of this role is a bit to be determined. As envisioned above, you could have fewer AVEs and have each of them be extremely high leverage. You might have a team of AVEs in a larger company, and they are embedded in a team or set of teams. Jade recommends starting with a person in each team, and if you prove that you can increase the amount of testing and validation done automatically, the role might eventually be one where an AVE works with a couple of teams at a time.

The AVE role doesn't mean that engineering doesn't have to 'own' quality. Automated verification engineers would be quality experts, similar to how Site Reliability Engineers are reliability experts.

SREs should not be doing all the reliability work, just like AVEs should not be doing all the quality work. The idea is that they bake quality into the system of work.

Shipping without human verification

Getting to production without a human in the loop is a target you build toward, the way teams build toward continuous deployment. Pick a percentage of commits and start small. One percent. Then work out what it would take for that slice to reach production with no human verification. It's the same question CI/CD answered for the deployment step: what set of practices do we need to safely ship something the moment it's approved? We built the rollback mechanisms and the monitoring that made that trust reasonable and stopped needing a person to watch each release.

the guardrails around them are real. The work is figuring out how to allowlist the safe changes and separate them from the risky ones, and that allowlisting is going to matter more and more.

Even without AI systems, you can probably get to 1%. You might find parts of your codebase where the risk of intent and quality drift are quite low. A simple CODEOWNERS change to make that part of the allowlist might be your first step on this journey. But try to track progress by increasing that percentage over time. What techniques and tools would you need to do that with confidence?

Building the verification system

As the volume of code increases, the bottleneck has moved to **code review**. Reviewers aren't able to keep up, and the bigger and bigger diffs will result in either rubber stamping or a role that is purely code review. Catching bugs through review was always an inefficient way to find them, and it never offered strong guarantees. The goal was always to reduce risk, not to eliminate it, because eliminating it entirely slows everything to a stop.

Automated verification good enough to ship without a human doesn't have to be perfect. It has to do better than what humans were doing, which is a lower bar than most teams admit.

What would automated verification look like in the case of code review? An **exercise you can run** is to review your team's last 1000 PR review comments. Sort them into deterministic, testable, and things that require judgment. Start adding automated guardrails for the first two, and figure out a mix of LLM and human judgment for the rest. The aim is to have a Swiss-cheese model, where you have multiple layers of assessments, or deterministic assessments, to verify you're getting reliable results out of your SDLC.

"Automated verification good enough to ship without a human doesn't have to be perfect. It has to do better than what humans were doing."

we're just starting to develop this as a practice. The ~~lean~~ **lean** of the **lean** codebase showed some of the challenges. Like CI/CD, you should probably make a goal to continuously increase the percentage of commits that can go to production without human review, but be rigorous about your standards. You need to have confidence you're making things better over time.

This approach goes way back: Andy Grove described this kind of quality control for chip manufacturing back in 1983: find where defects get through, build inspection around that point, and tighten the process as you learn. The same discipline is arriving in software: we're not building software anymore; we're building factories that build software.

Category	Lean	Example
Functional correctness/business logic	Test	<code>calculate_discount(cart)</code> returns \$18.00 for a 10%-off cart of \$20. A unit test pins the math; it should never need a human.
Edge cases & boundary conditions	Test	Empty cart, negative quantity, integer overflow, coupon expiring at midnight. Enumerable and deterministic — assert them all.
Known-bug regression protection	Test	Bug: refunds double-charged on payment retry. Add a test reproducing the retry path so it can never silently return.
API/schema/contract compatibility	Test	A contract test fails if a GraphQL field's type changes or a required response field is dropped.
Intent alignment	Verify	Ticket: "make checkout faster." The diff adds a cache — but does it address the slow path the ticket meant, or a cold path nobody hits? No assertion can answer that.
Acceptance-criteria coverage	Verify	AC lists: rate-limit login, log attempts, lock after 5 fails. The diff does the first and third but silently skips logging. Verify catches the missing item.
Architectural / convention adherence	Verify	Business logic added directly in a GraphQL resolver instead of the service layer. Tests

		primary button sits below the fold on mobile and the copy contradicts the design. Passes render tests; fails the intent.
Security review of new attack surface	Verify	A new endpoint accepts a <code>redirect_url</code> param. No test flags it, but Verify reasons: this is an open-redirect / SSRF surface – is it validated?
Side-effect / blast-radius completeness	Verify	Renamed a config key in code but didn't update the Helm chart, the docs, or the migration for existing values. What else does this change imply?
Non-deterministic/probabilistic behavior	Verify	An LLM-generated summary. You can't assert exact text – Verify judges whether it's faithful, complete, and on-tone.
Performance	Split	Test: a benchmark fails if p99 latency > 200ms. Verify: "this does an N+1 query in a loop – fine for 10 rows, will melt at 10k."

TNS



Jade Rubick builds thriving engineering organizations. Jade is a former VP of Engineering at New Relic and Gremlin. He writes Engineering Leadership Weekly, a newsletter on engineering leadership, and is an advisor and fractional VPE at many startups. Jade's interest...

[Read more from Jade Rubick →](#)



Ankit Jain is a cofounder and CEO of Aviator, a developer productivity platform used by modern engineering teams to ship AI-generated code at scale. He also leads The Hangar, a community of senior DevOps and senior software engineers focused on...

[Read more from Ankit Jain →](#)

TNS owner Insight Partners is an investor in: Anthropic.

RE THIS STORY

TRENDING STORIES

1. Shipping code without human verification
2. "Time to clean up human slop": Why AI now reviews code better than your teammate.
3. The 800 mistakes that could reshape Meta's AI coding strategy
4. Why a 'Boring' Database Is Your Secret AI Superpower
5. How to kill the code review

INSIGHTS FROM OUR SPONSOR



Aviator is a developer productivity platform used by modern engineering teams to ship AI-generated code at scale. Shared context, faster reviews, deterministic verification, and merge-to-deploy automation built for the volume AI creates.

[Learn More →](#)

Move code review before the code

3 August 2026

AI Code Verification vs AI Code Review: What's the Difference

23 July 2026

How to kill the code review

22 July 2026

How Verify Checks Code: Semantic Analysis, Runtime Previews, and Why It's Not Just Another AI Review Tool

21 July 2026

How long before we stop reading the code?

9 July 2026

TNS DAILY NEWSLETTER

Receive a free roundup of the
most recent TNS articles in
your inbox each day.

svo1975pnv1982@gmail.com

SUBSCRIBE

The New Stack does not sell your information or share it with unaffiliated third parties. By continuing, you agree to our [Terms of Use](#) and [Privacy Policy](#).

ARCHITECTURE

Cloud Native Ecosystem
Containers
Databases
Edge Computing
Infrastructure as Code
JVM
Linux

ENGINEERING

AI
AI Engineering
API Management
Backend development
Data
Frontend Development

[Networking](#)[Storage](#)[Software Development](#)[WebAssembly](#)

OPERATIONS

[AI Operations](#)[CI/CD](#)[Cloud Services](#)[DevOps](#)[Kubernetes](#)[Observability](#)[Operations](#)[Platform Engineering](#)

CHANNELS

[Podcasts](#)[Ebooks](#)[Events](#)[Webinars](#)[Newsletter](#)[TNS RSS Feeds](#)

THE NEW STACK

[About / Contact](#)[Sponsors](#)[Advertise With Us](#)[Contributions](#) [roadmap.sh](#)

Community created roadmaps, articles, resources and journeys for developers to help you choose your path and grow in your career.

[Frontend Developer Roadmap](#)[Backend Developer Roadmap](#)[Devops Roadmap](#)

FOLLOW TNS



© The New Stack 2026

[Disclosures](#)[Terms of Use](#)[Advertising Terms & Conditions](#)[Privacy Policy](#)[Cookie Policy](#)

